Guida Preliminare alla Preparazione dell'Esame di Machine Learning

Basato su dispense "La Sapienza" e testi d'esame

26 marzo 2026

Sommario

Questo documento è una guida completa e discorsiva per la preparazione all'esame di Machine Learning, basata sulle dispense del corso dell'Università "La Sapienza" e su un'analisi approfondita dei testi d'esame passati.

Indice

1 Introduzione al Machine Learning e Apprendimento di Concetti **4**
1.1 Tipi di Apprendimento . . . 4
1.2 Componenti di un Problema di Apprendimento . . . 4
1.3 Apprendimento di Concetti . . . 4
1.4 Spazio delle Ipotesi e Consistenza . . . 5

2 Valutazione delle Performance **5**
2.1 Definizioni di Errore . . . 5
2.2 Problemi nella Stima del True Error . . . 5
2.3 Intervalli di Confidenza . . . 5
2.4 K-Fold Cross Validation . . . 6
2.5 Matrice di Confusione e Metriche per la Classificazione . . . 6
2.6 Metriche per la Regressione . . . 7
2.7 Overfitting e Sottofitting . . . 7

3 Decision Trees (Alberi di Decisione) **7**
3.1 ID3 Algorithm . . . 8
3.2 Criteri di Scelta degli Attributi . . . 8
3.3 Overfitting e Pruning (Potatura) . . . 9

4 Probabilità e Reti Bayesiane **9**
4.1 Formule di Base della Probabilità . . . 9
4.2 Indipendenza Condizionale . . . 9
4.3 Reti Bayesiane . . . 9

5 Apprendimento Bayesiano **10**
5.1 Ipotesi MAP e ML . . . 10
5.2 Bayes Optimal Classifier (BOC) . . . 10
5.3 Naive Bayes Classifier . . . 10
5.4 Distribuzioni di Probabilità Comuni . . . 10
5.5 Naive Bayes per Classificazione di Testi . . . 11

6 Modelli Probabilistici per la Classificazione **11**
6.1 Modelli Generativi (es. Gaussian Mixture Model) . . . 11
6.2 Modelli Discriminativi (es. Logistic Regression) . . . 11

7 Modelli Lineari per la Classificazione **11**
7.1 Funzioni Discriminanti Lineari . . . 11
7.2 Metodi di Apprendimento . . . 12

8 Modelli Lineari per la Regressione **12**
8.1 Modelli Lineari a Funzione Base . . . 12
8.2 Maximum Likelihood e Least Squares . . . 13
8.3 Regularization (Regolarizzazione) . . . 13
8.4 Apprendimento Sequenziale (Stochastic Gradient Descent - SGD) . . . 13

9 Metodi Kernel **13**
9.1 Kernel Trick . . . 13
9.2 Funzioni Kernel Comuni . . . 13
9.3 Matrice Gramiana . . . 13
9.4 Kernelized SVM . . . 14

10 Apprendimento Basato su Istanza (Instance-Based Learning) **14**
10.1 K-Nearest Neighbors (K-NN) . . . 14

11 Reti Neurali Artificiali (ANN) **14**
11.1 Modello Generale Feedforward Network (FNN) . . . 14
11.2 Architettura e Funzioni di Attivazione . . . 14
11.3 Gradient Computation (Backpropagation) . . . 15
11.4 Training Algorithms . . . 15
11.5 Regularization (Tecniche Anti-Overfitting) . . . 15

12 Convolutional Neural Networks (CNN) **16**
12.1 Convoluzione . . . 16
12.2 Strati di una CNN . . . 16
12.3 CNN Famose . . . 16
12.4 Transfer Learning . . . 17

13 Apprendimento Multiplo (Ensemble Methods) **17**
13.1 Voting (Votazione) . . . 17
13.2 Bagging (Bootstrap Aggregating) . . . 17
13.3 Boosting (es. AdaBoost) . . . 17
13.4 Bias-Variance Decomposition . . . 18

14 Apprendimento Non Supervisionato **18**
14.1 K-Means Algorithm . . . 18
14.2 Gaussian Mixture Models (GMM) . . . 18

15 Riduzione della Dimensionalità **19**
15.1 Principal Component Analysis (PCA) . . . 19
15.2 Autoencoders (AE) . . . 19
15.3 Generative Adversarial Networks (GAN) . . . 19

16 Markov Decision Processes (MDP) e Reinforcement Learning (RL) **20**
16.1 Proprietà di Markov 20
16.2 MDP: Modello e Soluzione 20
16.3 Reinforcement Learning (RL) 21

17 Modelli Lineari per la Regressione **21**
17.1 Modelli Lineari a Funzione Base 21
17.2 Maximum Likelihood e Least Squares 21
17.3 Regularization (Regolarizzazione) 22
17.4 Apprendimento Sequenziale (Stochastic Gradient Descent - SGD) 22

18 Strategie per le Domande d'Esame **22**
18.1 Preparazione Generale 22
18.2 Approccio alle Domande 22
18.3 Temi Ricorrenti dagli Esami Forniti 23

1 Introduzione al Machine Learning e Apprendimento di Concetti

Il Machine Learning (ML) è una disciplina che permette ai computer di apprendere da dati o esperienza passata, migliorando le proprie prestazioni su un compito specifico senza essere esplicitamente programmati. È utile quando l'esperienza umana è limitata, la logica è troppo complessa da definire o le soluzioni devono adattarsi a casi specifici.

1.1 Tipi di Apprendimento

Il ML si divide principalmente in:

- **Supervised Learning (Apprendimento Supervisionato):** Il modello apprende da un dataset di esempi etichettati $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, dove $\mathbf{x}_i$ sono i dati di input e y_i sono le etichette di output desiderate. L'obiettivo è apprendere una funzione $f : \mathcal{X} \to \mathcal{Y}$ che possa prevedere correttamente l'output per nuovi input.
  - **Classificazione:** L'output $\mathcal{Y}$ è discreto (e.g., $\{0, 1\}$, {sì, no}, {gatto, cane})
  - **Regressione:** L'output $\mathcal{Y}$ è continuo (e.g., un valore numerico)
- **Unsupervised Learning (Apprendimento Non Supervisionato):** Il modello apprende da dati non etichettati $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$, cercando di scoprire pattern o strutture intrinseche nei dati.
  - **Clustering:** Raggruppare i dati in base alla loro somiglianza.
  - **Dimensionality Reduction (Riduzione della Dimensionalità):** Trovare una rappresentazione a dimensione inferiore dei dati, preservando il più possibile le informazioni rilevanti.
- **Reinforcement Learning (Apprendimento per Rinforzo):** Un agente apprende a prendere decisioni in un ambiente per massimizzare una ricompensa cumulativa. Non ci sono etichette esplicite, ma solo segnali di ricompensa/penalità.

1.2 Componenti di un Problema di Apprendimento

Ogni problema di apprendimento è definito da:

- **Compito (Task, $\mathcal{T}$):** Cosa si vuole che il sistema impari a fare (e.g., giocare a scacchi, riconoscere immagini).
- **Esperienza ($\mathcal{E}$):** I dati o le interazioni da cui il sistema apprende.
- **Misura di Performance ($\mathcal{P}$):** Come si valuta l'efficacia dell'apprendimento (e.g., accuratezza, tempo di vittoria).

1.3 Apprendimento di Concetti

Un concetto è una funzione binaria $c : \mathcal{X} \to \{0, 1\}$, dove 1 indica l'appartenenza al concetto (esempi positivi) e 0 la non appartenenza (esempi negativi). L'obiettivo è trovare un'ipotesi $h \in \mathcal{H}$ (spazio delle ipotesi) che approssimi al meglio c.
Rappresentazione di una Funzione Target Spesso, la funzione target è rappresentata come una combinazione lineare di feature del dominio. Ad esempio, per la valutazione di uno stato in un gioco:

$$\hat{V}(b) = w_0 + w_1 \cdot bp(b) + w_2 \cdot rp(b) + w_3 \cdot bk(b) + w_4 \cdot rk(b) + w_5 \cdot bt(b) + w_6 \cdot rt(b) \tag{1}$$

dove $bp(b), rp(b), \ldots$ sono feature del board b, e w_i sono i pesi da apprendere. L'algoritmo **LMS (Least Mean Squares) Weight Update Rule** è un modo comune per aggiornare questi pesi:

$$w_i \leftarrow w_i + c \cdot f_i \cdot error(b) \tag{2}$$

dove c è il learning rate, f_i è la i-esima feature e $error(b)$ è l'errore di predizione.

1.4 Spazio delle Ipotesi e Consistenza

Spazio delle Ipotesi ($\mathcal{H}$): L'insieme di tutte le possibili funzioni che l'algoritmo può apprendere. Un'ipotesi h è **consistente** con un dataset di training $\mathcal{D}$ se $h(\mathbf{x}) = c(\mathbf{x})$ per tutti gli esempi $(\mathbf{x}, c(\mathbf{x})) \in \mathcal{D}$. **Version Space ($VS_{\mathcal{H},\mathcal{D}}$):** L'insieme di tutte le ipotesi in $\mathcal{H}$ che sono consistenti con $\mathcal{D}$.

$$VS_{\mathcal{H},\mathcal{D}} \equiv \{h \in \mathcal{H} | Consistent(h, \mathcal{D})\} \tag{3}$$

Gli algoritmi come LIST-THEN-ELIMINATE mirano a trovare tutte le ipotesi consistenti.

2 Valutazione delle Performance

La valutazione delle performance di un modello di ML è cruciale per capirne l'efficacia e generalizzabilità.

2.1 Definizioni di Errore

True Error ($error_{\mathcal{D}}(h)$): La probabilità che l'ipotesi h misclassifichi un'istanza casuale estratta dalla distribuzione $\mathcal{D}$.

$$error_{\mathcal{D}}(h) \equiv \Pr_{x \in \mathcal{D}}[f(x) \neq h(x)] \tag{4}$$

Sample Error ($error_S(h)$): La proporzione di esempi che h misclassifica nel dataset di training S.

$$error_S(h) \equiv \frac{1}{n} \sum_{x \in S} \delta(f(x) \neq h(x)) \tag{5}$$

dove $\delta(\cdot)$ è 1 se l'argomento è vero, 0 altrimenti.

2.2 Problemi nella Stima del True Error

Il *trueerror* non può essere calcolato direttamente perché $\mathcal{D}$ è sconosciuta. Il *sampleerror* è una stima, ma può essere:

- **Bias Ottimistico:** Se S è lo stesso set usato per addestrare h, $error_S(h)$ sarà una stima ottimistica di $error_{\mathcal{D}}(h)$.
- **Varianza:** Se S è troppo piccolo, $error_S(h)$ può variare significativamente.

Estimation Bias:

$$bias \equiv E_S[error_S(h)] - error_{\mathcal{D}}(h) \tag{6}$$

2.3 Intervalli di Confidenza

Per stimare $error_{\mathcal{D}}(h)$ in modo non distorto, h e S devono essere scelti indipendentemente. Per $n \geq 30$, con una probabilità $N\%$ (e.g., 95%), $error_{\mathcal{D}}(h)$ si trova nell'intervallo:

$$error_S(h) \pm z_N \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}} \tag{7}$$

dove z_N dipende dal livello di confidenza.

2.4 K-Fold Cross Validation

Una tecnica robusta per stimare la performance di un algoritmo di apprendimento L su un dataset $\mathcal{D}$:

1. Partizionare $\mathcal{D}$ in k sottoinsiemi disgiunti $S_1, \ldots, S_k$.
2. Per $i = 1, \ldots, k$:
   - Usare S_i come set di test e $T_i = \mathcal{D} \setminus S_i$ come set di training.
   - Addestrare $h_i = L(T_i)$.
   - Calcolare $\delta_i = error_{S_i}(h_i)$.
3. L'errore stimato è la media dei δ_i:

$$error_{L,\mathcal{D}} \equiv \frac{1}{k} \sum_{i=1}^{k} \delta_i \tag{8}$$

Questo metodo può essere esteso per confrontare due algoritmi L_A e L_B.

2.5 Matrice di Confusione e Metriche per la Classificazione

La **Matrice di Confusione** è una tabella che riassume le performance di un algoritmo di classificazione. Per una classificazione binaria (classi Positiva P, Negativa N):

- **True Positive (TP):** Predetto P, Vero P.
- **True Negative (TN):** Predetto N, Vero N.
- **False Positive (FP):** Predetto P, Vero N (errore di tipo I).
- **False Negative (FN):** Predetto N, Vero P (errore di tipo II).

Da essa si derivano:

- **Error Rate:** Proporzione di misclassificazioni.

$$\text{Error rate} = \frac{FN + FP}{TP + TN + FP + FN} \tag{9}$$

- **Accuracy:** Proporzione di classificazioni corrette.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{10}$$

- **Recall (Sensibilità, True Positive Rate - TPR):** Capacità di trovare tutti gli esempi positivi.

$$\text{Recall} = \text{TPR} = \frac{TP}{P} = \frac{TP}{TP + FN} \tag{11}$$

- **Specificity (True Negative Rate - TNR):** Capacità di trovare tutti gli esempi negativi.

$$\text{TNR} = \frac{TN}{N} = \frac{TN}{TN + FP} \tag{12}$$

- **Precision:** Proporzione di predizioni positive corrette.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{13}$$

- **F1-Score:** Media armonica tra Precision e Recall. Utile per dataset sbilanciati.

$$\text{F1-score} = 2\frac{Precision \cdot Recall}{Precision + Recall} \tag{14}$$

- **False Positive Rate (FPR):** Proporzione di negativi classificati erroneamente come positivi.

$$\text{FPR} = \frac{FP}{N} = \frac{FP}{TN + FP} \tag{15}$$

- **False Negative Rate (FNR):** Proporzione di positivi classificati erroneamente come negativi.

$$\text{FNR} = \frac{FN}{P} = \frac{FN}{TP + FN} \tag{16}$$

Quando l'Accuracy non basta: Per dataset sbilanciati (e.g., rilevazione di anomalie, malattie rare), l'accuracy può essere fuorviante. Precision, Recall e F1-Score offrono una visione più completa.

2.6 Metriche per la Regressione

- **Mean Absolute Error (MAE):** Media degli errori assoluti.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |\hat{f}(x_i) - t_i| \tag{17}$$

- **Mean Squared Error (MSE):** Media degli errori al quadrato, penalizza maggiormente gli errori grandi.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} (\hat{f}(x_i) - t_i)^2 \tag{18}$$

- **Root Mean Squared Error (RMSE):** Radice quadrata dell'MSE, nella stessa unità di misura dell'output.

$$\text{RMSE} = \sqrt{\text{MSE}} \tag{19}$$

- **Coefficiente di Determinazione (R^2):** Misura la proporzione della varianza nell'output prevedibile dal modello. $R^2 = 1$ indica predizione perfetta, $R^2 = 0$ predizione uguale alla media.

$$R^2 = 1 - \frac{\sum_{n=1}^{N}(y_n - t_n)^2}{\sum_{n=1}^{N}(t_n - \mu)^2} \tag{20}$$

2.7 Overfitting e Sottofitting

Overfitting: Un modello si adatta troppo bene ai dati di training, imparando anche il rumore. Risulta in alta accuratezza sul training set ma bassa sulla generalizzazione a nuovi dati.

- $error_S(h) < error_S(h')$
- $error_{\mathcal{D}}(h) > error_{\mathcal{D}}(h')$

Sottofitting (Underfitting): Un modello è troppo semplice per catturare la struttura sottostante dei dati. Risulta in bassa accuratezza sia sul training che sul test set.

3 Decision Trees (Alberi di Decisione)

I Decision Trees sono modelli non parametrici usati per classificazione e regressione, che rappresentano le decisioni in modo esplicito e interpretabile.

3.1 ID3 Algorithm

ID3 (Iterative Dichotomiser 3) è un algoritmo per costruire alberi di decisione.

- **Input:** Esempi di training, Attributi disponibili, Attributo Target.
- **Output:** Un albero di decisione.
- **Passi Principali:**
  1. Crea un nodo radice per l'albero.
  2. Se tutti gli esempi sono positivi, il nodo radice diventa una foglia etichettata '+'.
  3. Se tutti gli esempi sono negativi, il nodo radice diventa una foglia etichettata '-'.
  4. Se non ci sono più attributi disponibili, il nodo radice diventa una foglia etichettata con la classe più comune tra gli esempi.
  5. Altrimenti, seleziona l'attributo A che massimizza il **Guadagno di Informazione (Information Gain)**.
  6. Il nodo radice diventa un nodo di test per l'attributo A.
  7. Per ogni valore v_i dell'attributo A:
     - Crea un nuovo ramo per il nodo radice corrispondente al test $A = v_i$.
     - Filtra gli esempi di training per includere solo quelli con $A = v_i$.
     - Applica ricorsivamente ID3 a questo sottoinsieme di esempi e attributi rimanenti.

3.2 Criteri di Scelta degli Attributi

La scelta dell'attributo migliore per splittare i dati è fondamentale:

- **Entropia ($Entropy(S)$):** Misura l'impurità o l'incertezza in un insieme di dati S. Per classi binarie $(p_\oplus, p_\ominus)$:

$$Entropy(S) \equiv -p_\oplus \log_2 p_\oplus - p_\ominus \log_2 p_\ominus \tag{21}$$

  Per classi multiple:

$$Entropy(S) \equiv \sum_{i=1}^{c} -p_i \log_2 p_i \tag{22}$$

- **Information Gain ($Gain(S, A)$):** La riduzione attesa di entropia ottenuta splittando l'insieme S usando l'attributo A. Si sceglie l'attributo con il $Gain$ più alto.

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v) \tag{23}$$

- **Gain Ratio:** Utilizzato per contrastare il bias dell'Information Gain verso attributi con molti valori.

$$SplitInformation(S, A) \equiv -\sum_{i=1}^{c} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|} \tag{24}$$

$$GainRatio(S, A) \equiv \frac{Gain(S, A)}{SplitInformation(S, A)} \tag{25}$$

3.3 Overfitting e Pruning (Potatura)

Gli alberi di decisione sono soggetti a overfitting, specialmente se crescono troppo in profondità. Il **Pruning** è una tecnica per ridurre la complessità dell'albero e migliorare la generalizzazione.

- **Pre-Pruning:** Fermare la crescita dell'albero prematuramente (e.g., quando il guadagno di informazione è sotto una certa soglia, o il numero di esempi in un nodo è troppo basso).
- **Post-Pruning (e.g., Reduced-Error Pruning):**
  1. Costruire un albero completo (potenzialmente in overfitting).
  2. Dividere i dati in training e validation set.
  3. Per ogni nodo interno, valutare l'effetto di sostituire il sottoalbero con una foglia (etichettata con la classe più comune del sottoalbero) sul validation set.
  4. Sostituire iterativamente il sottoalbero che offre il maggior miglioramento (o minor deterioramento) della performance sul validation set, fino a quando non si osservano ulteriori benefici.
- **Rule Post-Pruning (e.g., C4.5):** Convertire l'albero in un insieme di regole, potare (generalizzare) ogni regola indipendentemente, e poi ordinarle. Questo può portare a migliore generalizzazione.

4 Probabilità e Reti Bayesiane

Le probabilità forniscono un linguaggio formale per ragionare sull'incertezza. Le reti Bayesiane sono modelli grafici che rappresentano relazioni di dipendenza condizionale tra variabili.

4.1 Formule di Base della Probabilità

- **Probabilità Condizionale:** $P(A|B) = \frac{P(A \wedge B)}{P(B)}$ se $P(B) \neq 0$.
- **Regola del Prodotto:** $P(A \wedge B) = P(A|B)P(B) = P(B|A)P(A)$.
- **Regola della Somma (Marginalizzazione):** $P(X) = \sum_{y \in \mathcal{D}(Y)} P(X, Y = y)$.
- **Teorema di Bayes:** $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$.
- **Regola della Catena:** $P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i|X_1, \ldots, X_{i-1})$.

4.2 Indipendenza Condizionale

X è condizionalmente indipendente da Y dato Z se $P(X|Y, Z) = P(X|Z)$. Questo riduce la complessità della rappresentazione delle distribuzioni congiunte.

$$P(X, Y|Z) = P(X|Y, Z)P(Y|Z) = P(X|Z)P(Y|Z) \tag{26}$$

4.3 Reti Bayesiane

- **Sintassi:** Un grafo diretto aciclico (DAG) dove i nodi sono variabili e gli archi rappresentano "influenze dirette". Ogni nodo X_i ha una distribuzione di probabilità condizionale $P(X_i|Parents(X_i))$.
- **Semantica:** La distribuzione congiunta di tutte le variabili è data da:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i|Parents(X_i)) \tag{27}$$

- **Compattezza:** Permettono una specificazione compatta delle distribuzioni congiunte, sfruttando l'indipendenza condizionale.

5 Apprendimento Bayesiano

L'apprendimento Bayesiano integra la conoscenza a priori con l'evidenza dei dati per formare nuove credenze.

5.1 Ipotesi MAP e ML

Dato un dataset D e uno spazio di ipotesi $\mathcal{H}$:

- **Ipotesi Maximum A Posteriori (h_{MAP}):** L'ipotesi più probabile dato il dataset.

$$h_{MAP} \equiv \arg\max_{h \in \mathcal{H}} P(h|D) = \arg\max_{h \in \mathcal{H}} P(D|h)P(h) \tag{28}$$

- **Ipotesi Maximum Likelihood (h_{ML}):** L'ipotesi che rende più probabile l'osservazione del dataset D. Si usa quando non si ha conoscenza a priori (o si assume $P(h)$ uniforme).

$$h_{ML} = \arg\max_{h \in \mathcal{H}} P(D|h) \tag{29}$$

5.2 Bayes Optimal Classifier (BOC)

Il BOC fornisce la classificazione più probabile per una nuova istanza x', considerando tutte le possibili ipotesi in $\mathcal{H}$, pesate dalla loro probabilità a posteriori.

$$v_{OB} = \arg\max_{v_j \in V} \sum_{h_i \in \mathcal{H}} P(v_j|x', h_i)P(h_i|D) \tag{30}$$

È l'ottimo teorico, ma spesso computazionalmente intrattabile se $\mathcal{H}$ è troppo grande.

5.3 Naive Bayes Classifier

Il Naive Bayes semplifica il BOC assumendo **indipendenza condizionale** delle feature dato la classe: $P(a_1, \ldots, a_n|v_j, D) \approx \prod_i P(a_i|v_j, D)$. **Classificazione per Naive Bayes:**

$$v_{NB} = \arg\max_{v_j \in V} P(v_j|D) \prod_i P(a_i|v_j, D) \tag{31}$$

Stima delle Probabilità (Laplace Smoothing): Per evitare probabilità zero, si usa lo smoothing di Laplace:

$$\hat{P}(a_i|v_j, D) = \frac{|\{\ldots, a_i, \ldots, v_j\}| + mp}{|\{\ldots, v_j\}| + m} \tag{32}$$

dove p è una stima a priori per $P(a_i|v_j, D)$ e m è un peso (numero di esempi virtuali). **Assunzione di Indipendenza:** Sebbene l'assunzione di indipendenza sia spesso violata nella realtà, Naive Bayes si comporta sorprendentemente bene in pratica, specialmente per la classificazione di testi.

5.4 Distribuzioni di Probabilità Comuni

- **Bernoulli:** Per variabili binarie (0 o 1). $P(X = k; \theta) = \theta^k (1-\theta)^{1-k}$.
- **Multivariata Bernoulli:** Per vettori di variabili binarie indipendenti.

$$P(X_1 = k_1, \ldots, X_n = k_n; \theta_1, \ldots, \theta_n) = \prod_{i=1}^{n} \theta_i^{k_i} (1-\theta_i)^{1-k_i} \tag{33}$$

- **Binomiale:** Per k successi in n prove Bernoulli. $P(X = k; n, \theta) = \binom{n}{k} \theta^k (1-\theta)^{n-k}$.
- **Multinomiale:** Per k_j occorrenze di d risultati diversi in n prove.

$$P(X_1 = k_1, \ldots, X_d = k_d; n, \theta_1, \ldots, \theta_d) = \frac{n!}{k_1! \ldots k_d!} \theta_1^{k_1} \ldots \theta_d^{k_d} \tag{34}$$

5.5 Naive Bayes per Classificazione di Testi

Per classificare documenti, si usa la Naive Bayes, assumendo che le parole nel documento siano condizionalmente indipendenti data la classe del documento.

$$c_{NB} = \arg\max_{c_j \in C} P(c_j|D) \prod_{i=1}^{\text{length}(doc)} P(w_i|c_j) \tag{35}$$

Tecniche come lo stemming e la rimozione di stop words sono usate per pre-processare il testo.

6 Modelli Probabilistici per la Classificazione

Questi modelli cercano di apprendere le probabilità di classe $P(C_k|\mathbf{x})$.

6.1 Modelli Generativi (es. Gaussian Mixture Model)

Stimano $P(\mathbf{x}|C_k)$ e $P(C_k)$, poi usano il teorema di Bayes per trovare $P(C_k|\mathbf{x})$. Per 2 classi e distribuzioni Gaussiane con stessa covarianza Σ:

$$P(C_1|\mathbf{x}) = \sigma(a) = \frac{1}{1 + \exp(-a)} \tag{36}$$

dove $a = \ln \frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x}|C_2)P(C_2)} = \mathbf{w}^T\mathbf{x} + w_0$. I parametri $\mathbf{w}$ e w_0 dipendono dalle medie μ_1, μ_2, dalla covarianza Σ e dalle probabilità a priori π.

6.2 Modelli Discriminativi (es. Logistic Regression)

Stimano $P(C_k|\mathbf{x})$ direttamente. **Logistic Regression:** Per 2 classi, usa una funzione sigmoide:

$$P(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + w_0) \tag{37}$$

Per K classi, usa la funzione softmax:

$$P(C_k|\mathbf{x}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \quad \text{dove } a_k = \mathbf{w}_k^T\mathbf{x} + w_{k0} \tag{38}$$

Il training avviene minimizzando la funzione di costo **Cross-Entropy** (log-likelihood negativa). Per 2 classi:

$$E(\tilde{\mathbf{w}}) = -\sum_{n=1}^{N} [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)] \tag{39}$$

L'ottimizzazione spesso avviene con metodi iterativi come **Iterative Reweighted Least Squares (IRLS)**, che è una forma di Newton-Raphson:

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \mathbf{H}(\tilde{\mathbf{w}})^{-1} \nabla E(\tilde{\mathbf{w}}) \tag{40}$$

7 Modelli Lineari per la Classificazione

I modelli lineari cercano di trovare un iperpiano che separi le classi nello spazio delle feature.

7.1 Funzioni Discriminanti Lineari

L'output è una combinazione lineare delle feature:

$$y(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 = \tilde{\mathbf{w}}^T\tilde{\mathbf{x}} \tag{41}$$

Per K classi, si hanno K funzioni discriminanti: $y_k(\mathbf{x}) = \mathbf{w}_k^T\mathbf{x} + w_{k0}$. La classe predetta è quella con il valore $y_k(\mathbf{x})$ più alto.

7.2 Metodi di Apprendimento

- **Least Squares:** Minimizza l'errore quadratico tra l'output desiderato e quello del modello. La soluzione in forma chiusa è:

$$\mathbf{W}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{T} \tag{42}$$

  Non robusto agli outlier, assume distribuzioni Gaussiane.

- **Perceptron:** Un algoritmo iterativo che aggiorna i pesi quando un esempio è misclassificato.
  - Output: $o(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x})$.
  - Regola di aggiornamento: $\Delta w_i = \eta(t_n - o_n)x_{i,n}$.

  Con dati linearmente separabili e η sufficientemente piccolo, converge. Non robusto agli outlier (l'iperpiano può essere troppo vicino a una classe).

- **Fisher's Linear Discriminant:** Trova una direzione $\mathbf{w}$ che massimizza la separazione tra le medie delle classi e minimizza la varianza intra-classe.

$$J(\mathbf{w}) = \frac{\mathbf{w}^T\mathbf{S}_B\mathbf{w}}{\mathbf{w}^T\mathbf{S}_W\mathbf{w}} \tag{43}$$

  La soluzione è $\mathbf{w}^* \propto \mathbf{S}_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$.

- **Support Vector Machines (SVM):** Cerca l'iperpiano che massimizza il **margine** tra le classi, rendendolo robusto agli outlier.
  - **Problema di Ottimizzazione (Hard Margin):**

$$\min_{\mathbf{w},w_0} \frac{1}{2}||\mathbf{w}||^2 \quad \text{s.t.} \quad t_n(\mathbf{w}^T\mathbf{x}_n + w_0) \geq 1 \quad \forall n \tag{44}$$

  - **Soft Margin (con Outlier):** Introduce variabili di slack $\xi_n \geq 0$ per permettere alcune violazioni del margine.

$$\min_{\mathbf{w},w_0,\boldsymbol{\xi}} \frac{1}{2}||\mathbf{w}||^2 + C\sum_{n=1}^{N}\xi_n \quad \text{s.t.} \quad t_n(\mathbf{w}^T\mathbf{x}_n + w_0) \geq 1 - \xi_n, \quad \xi_n \geq 0 \quad \forall n \tag{45}$$

  - **Vettori di Supporto:** Solo gli esempi più vicini all'iperpiano (o che violano il margine) contribuiscono alla definizione dell'iperpiano.

8 Modelli Lineari per la Regressione

Cercano di approssimare una funzione continua $f : \mathcal{X} \to \mathcal{R}$.

8.1 Modelli Lineari a Funzione Base

$y(\mathbf{x};\mathbf{w}) = \sum_{j=0}^{M} w_j\phi_j(\mathbf{x}) = \mathbf{w}^T\phi(\mathbf{x})$ dove $\phi_j(\mathbf{x})$ sono funzioni base non lineari (es. polinomiali, radiali, sigmoidali) che trasformano lo spazio di input in uno spazio feature. Il modello rimane lineare nei parametri w.

8.2 Maximum Likelihood e Least Squares

Assumendo rumore Gaussiano sull'output $t = y(\mathbf{x}; \mathbf{w}) + \epsilon$, l'ottimizzazione dei parametri w si riduce alla minimizzazione dell'errore quadratico (Least Squares):

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} [t_n - \mathbf{w}^T \phi(\mathbf{x}_n)]^2 \tag{46}$$

La soluzione in forma chiusa è:

$$\mathbf{w}_{ML} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t} \tag{47}$$

dove Φ è la design matrix.

8.3 Regularization (Regolarizzazione)

Tecnica per prevenire l'overfitting, aggiungendo un termine di penalità alla funzione di errore.

$$\min_{\mathbf{w}} E_D(\mathbf{w}) + \lambda E_W(\mathbf{w}) \tag{48}$$

dove $E_W(\mathbf{w})$ penalizza la complessità del modello (es. L_1 o L_2 norm dei pesi).

8.4 Apprendimento Sequenziale (Stochastic Gradient Descent - SGD)

Metodo iterativo per grandi dataset. L'aggiornamento avviene per ogni esempio o per mini-batch: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_S$ dove η è il learning rate. Varianti come SGD con momentum migliorano la convergenza.

9 Metodi Kernel

Estendono i modelli lineari a funzioni non lineari, operando in uno spazio feature ad alta dimensione (anche infinita) senza calcolare esplicitamente la trasformazione $\phi(\mathbf{x})$.

9.1 Kernel Trick

Se un algoritmo utilizza i vettori di input $\mathbf{x}$ solo sotto forma di prodotto interno $\mathbf{x}^T \mathbf{x}'$, possiamo sostituire questo prodotto interno con una **funzione kernel** $k(\mathbf{x}, \mathbf{x}')$. Questo ci permette di operare implicitamente nello spazio feature $\phi(\mathbf{x})$ dove $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}')$. **Condizioni per un Kernel Valido:**

- Simmetria: $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x})$
- Non-negatività (la matrice Gramiana deve essere semi-definita positiva).

9.2 Funzioni Kernel Comuni

- **Lineare:** $k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$.
- **Polinomiale:** $k(\mathbf{x}, \mathbf{x}') = (\beta \mathbf{x}^T \mathbf{x}' + \gamma)^d$.
- **Radial Basis Function (RBF) / Gaussiano:** $k(\mathbf{x}, \mathbf{x}') = \exp(-\beta ||\mathbf{x} - \mathbf{x}'||^2)$.
- **Sigmoide:** $k(\mathbf{x}, \mathbf{x}') = \tanh(\beta \mathbf{x}^T \mathbf{x}' + \gamma)$.

9.3 Matrice Gramiana

La **Matrice Gramiana K** è una matrice $N \times N$ dove $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. È un elemento chiave nei metodi kernel. **Kernelized Linear Models:** La soluzione dei modelli lineari in termini di α spesso coinvolge la matrice Gramiana. Ad esempio, per regressione: $\alpha = (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t}$ La predizione diventa: $y(\mathbf{x}) = \sum_{n=1}^{N} \alpha_n k(\mathbf{x}_n, \mathbf{x})$.

9.4 Kernelized SVM

Nel contesto SVM, il kernel trick è applicato per risolvere problemi non linearmente separabili nello spazio originale. Il problema di ottimizzazione duale coinvolge direttamente il kernel: $\mathbf{w}^* = \sum_{n=1}^{N} a_n^* t_n \phi(\mathbf{x}_n)$, la predizione diventa $y(\mathbf{x}) = \sum_{n=1}^{N} a_n^* t_n k(\mathbf{x}_n, \mathbf{x}) + w_0^*$. **Kernelized SVM Regression:** Estende SVM per la regressione, usando un ϵ-insensitive error function.

10 Apprendimento Basato su Istanza (Instance-Based Learning)

Questi modelli non apprendono un modello esplicito durante il training, ma memorizzano tutti i dati di training e fanno previsioni basate sulla somiglianza con nuove istanze.

10.1 K-Nearest Neighbors (K-NN)

- **Classificazione:** Per una nuova istanza $\mathbf{x}$, trova i K vicini più prossimi nel training set. Assegna a $\mathbf{x}$ la classe più comune tra questi K vicini.

$$p(c|\mathbf{x}, D, K) = \frac{1}{K} \sum_{\mathbf{x}_n \in N_K(\mathbf{x}, D)} \mathbb{I}(t_n = c) \tag{49}$$

- **Regressione:** Per una nuova istanza $\mathbf{x}$, la predizione è la media dei valori target dei K vicini.
- **Funzione di Distanza:** La scelta della metrica di distanza (es. Euclidea) è fondamentale.
- **K:** Un valore basso di K rende il modello sensibile al rumore e a overfitting; un valore alto di K leviga i confini decisionali ma può causare underfitting.

Kernelized K-NN: Le funzioni di distanza possono essere kernelizzate. **Svantaggi:** Richiede la memorizzazione di tutto il dataset, computazionalmente costoso per dataset grandi.

11 Reti Neurali Artificiali (ANN)

Le ANN sono modelli parametrici ispirati alla struttura del cervello, capaci di approssimare funzioni complesse.

11.1 Modello Generale Feedforward Network (FNN)

Una FNN è una composizione di strati di neuroni. Ogni neurone in uno strato calcola una combinazione lineare degli output dello strato precedente e applica una funzione di attivazione non lineare.

$$\mathbf{h}_{i+1} = f(\mathbf{W}_i^T \mathbf{h}_i + \mathbf{b}_i) \tag{50}$$

dove $\mathbf{h}_i$ è l'output dello strato i, $\mathbf{W}_i$ è la matrice dei pesi, $\mathbf{b}_i$ è il bias, e f è la funzione di attivazione. **Parametri Addestrabili:** I pesi $\mathbf{W}_i$ e i bias $\mathbf{b}_i$.

11.2 Architettura e Funzioni di Attivazione

- **Profondità (Depth):** Numero di strati nascosti. Reti "profonde" sono la base del Deep Learning.
- **Ampiezza (Width):** Numero di unità (neuroni) per strato.
- **Funzioni di Attivazione (Activation Functions):** Introducono non linearità, permettendo alla rete di apprendere relazioni complesse.

- **ReLU (Rectified Linear Unit):** $g(\alpha) = \max(0, \alpha)$. Veloce da calcolare, non satura per valori positivi.
- **Sigmoide:** $g(\alpha) = \sigma(\alpha) = \frac{1}{1+\exp(-\alpha)}$. Output tra 0 e 1, usata spesso per output binari. Satura (gradienti piccoli).
- **Tanh (Hyperbolic Tangent):** $g(\alpha) = \tanh(\alpha) = 2\sigma(2\alpha) - 1$. Output tra -1 e 1, centrato a 0. Satura.

- **Unità di Output e Funzioni di Costo:**
  - **Regressione (output lineare):** $y = \mathbf{W}^T\mathbf{h} + \mathbf{b}$. Loss: Mean Squared Error (MSE).
  - **Classificazione Binaria (output sigmoide):** $y = \sigma(\mathbf{w}^T\mathbf{h} + b)$. Loss: Binary Cross-Entropy.
  - **Classificazione Multi-Classe (output softmax):** $y_i = \text{softmax}(\alpha^{(i)}) = \frac{\exp(\alpha^{(i)})}{\sum_j \exp(\alpha_j)}$. Loss: Categorical Cross-Entropy.

11.3 Gradient Computation (Backpropagation)

L'algoritmo di **Backpropagation** è il metodo standard per calcolare i gradienti della funzione di costo rispetto ai pesi della rete, sfruttando la regola della catena.

$$\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^T \nabla_{\mathbf{y}} z \tag{51}$$

- **Fase Forward:** L'input attraversa la rete, calcolando gli output di ogni strato fino all'output finale.
- **Fase Backward:** Il gradiente dell'errore viene propagato all'indietro attraverso la rete, calcolando i gradienti per ogni peso e bias.

11.4 Training Algorithms

- **Stochastic Gradient Descent (SGD):** Aggiorna i pesi usando il gradiente calcolato su un singolo esempio o un mini-batch di esempi. $\theta^{(k+1)} \leftarrow \theta^{(k)} - \eta\mathbf{g}$
- **SGD with Momentum:** Aggiunge un termine di momentum per accelerare la convergenza e ridurre le oscillazioni. $\mathbf{v}^{(k+1)} \leftarrow \mu\mathbf{v}^{(k)} - \eta\mathbf{g}$
  $\theta^{(k+1)} \leftarrow \theta^{(k)} + \mathbf{v}^{(k+1)}$
- **SGD with Nesterov Momentum:** Un'evoluzione del momentum.
- **Algoritmi con Learning Rate Adattivo:** (e.g., AdaGrad, RMSProp, Adam) Adattano il learning rate per ogni parametro, spesso migliorando la convergenza.

11.5 Regularization (Tecniche Anti-Overfitting)

- **Parameter Norm Penalties (L1/L2 Regularization):** Aggiunge un termine alla loss function che penalizza la grandezza dei pesi. $\bar{J}(\theta) = J(\theta) + \lambda \sum_j |\theta_j|^q$ (con $q = 1$ per L1, $q = 2$ per L2).
- **Dataset Augmentation:** Aumenta la quantità di dati di training con trasformazioni (e.g., rotazione, scaling di immagini) o aggiungendo rumore.
- **Early Stopping:** Ferma il training quando la performance sul validation set inizia a peggiorare, anche se quella sul training set continua a migliorare.
- **Dropout:** Durante il training, ignora casualmente (setta a zero) un sottoinsieme di neuroni con una certa probabilità. Questo previene che la rete si affidi eccessivamente a singoli neuroni.

12 Convolutional Neural Networks (CNN)

Le CNN sono un tipo specializzato di ANN, particolarmente efficaci per dati con struttura griglia (e.g., immagini, video, audio), sfruttando convoluzioni.

12.1 Convoluzione

L'operazione fondamentale.

- **Continua:** $(x * w)(t) \equiv \int_{-\infty}^{\infty} x(a)w(t-a)da$
- **Discreta (2D):** $(I * K)(i,j) \equiv \sum_{m \in S_1} \sum_{n \in S_2} I(m,n)K(i-m, j-n)$

Nelle librerie ML, si implementa spesso la **cross-correlation** per efficienza, che è simile ma senza il flipping del kernel: $(I * K)(i,j) = \sum_{m \in S_1} \sum_{n \in S_2} I(i+m, j+n)K(m,n)$

12.2 Strati di una CNN

Una CNN tipica alterna strati convoluzionali, di attivazione e di pooling.

- **Input Layer:** Immagine di input (e.g., $W_{in} \times H_{in} \times D_{in}$ dimensioni).
- **Strato Convoluzionale (Convolutional Layer):**
  - Applicazione di più **kernel** (filtri) all'input. Ogni kernel genera una **feature map**.
  - **Kernel Size ($W_k \times H_k \times D_k$):** Dimensioni del filtro.
  - **Stride (s):** Passo con cui il kernel scorre sull'input.
  - **Padding (p):** Aggiunta di valori (tipicamente zero) ai bordi per controllare la dimensione dell'output.
  - **Output Feature Map Size:** $W_{out} = \frac{W_{in}-W_k+2p}{s} + 1$, $H_{out} = \frac{H_{in}-H_k+2p}{s} + 1$.
  - **Numero di Parametri Addestrabili:** Ogni kernel ha pesi e un bias. $W_k \cdot H_k \cdot D_{in} \cdot D_{out} + D_{out}$.
  - **Sparse Connectivity (Connettività Sparsa):** Ogni neurone di output è connesso solo a una piccola regione dell'input (receptive field), non a tutto l'input.
  - **Parameter Sharing (Condivisione di Parametri):** Lo stesso kernel è applicato a diverse posizioni spaziali dell'input, riducendo il numero di parametri e introducendo invarianza alla traslazione.
- **Strato di Attivazione (Detector Layer):** Applica una funzione di attivazione non lineare (spesso ReLU) ad ogni elemento della feature map.
- **Strato di Pooling (Pooling Layer):** Riduce la dimensione spaziale della rappresentazione per ridurre la computazione e il numero di parametri, e per ottenere invarianza a piccole traslazioni.
  - **Max Pooling:** Seleziona il valore massimo in una regione.
  - **Average Pooling:** Calcola la media in una regione.
- **Fully Connected Layer:** Dopo diversi strati convoluzionali e di pooling, i dati vengono "appiattiti" (flatten) e passati a uno o più strati completamente connessi, come in una ANN tradizionale, per la classificazione o regressione finale.

12.3 CNN Famose

Architetture come LeNet, AlexNet, VGG, GoogLeNet/Inception e ResNet hanno segnato pietre miliari nel campo, mostrando performance sempre migliori (e.g., in ILSVRC).

12.4 Transfer Learning

Riutilizzare un modello pre-addestrato (e.g., su ImageNet) per un nuovo compito o dominio, spesso con dataset più piccoli.

- **Fine-tuning:** Adattare i pesi di un modello pre-addestrato sul nuovo dataset. Si può addestrare l'intera rete o solo gli ultimi strati.
- **CNN come Feature Extractor:** Usare i primi strati della CNN pre-addestrata per estrarre feature, e poi addestrare un nuovo classificatore (e.g., SVM, rete neurale più piccola) su queste feature.

13 Apprendimento Multiplo (Ensemble Methods)

Gli ensemble methods combinano più learner (modelli) per ottenere una performance migliore rispetto a un singolo modello.

13.1 Voting (Votazione)

Combina le predizioni di più modelli addestrati in parallelo.

- **Regressione:** Media pesata degli output. $y_{voting}(\mathbf{x}) = \sum_{m=1}^{M} w_m y_m(\mathbf{x})$.
- **Classificazione:** Maggioranza pesata delle classi predette. $y_{voting}(\mathbf{x}) = \arg\max_c \sum_{m=1}^{M} w_m \mathbb{I}(y_m(\mathbf{x}) = c)$.

13.2 Bagging (Bootstrap Aggregating)

Addestra M modelli indipendenti su M sottoinsiemi di dati campionati con replacement (bootstrap) dal dataset originale.

- **Training:** Genera M dataset bootstrap $\mathcal{D}_1, \ldots, \mathcal{D}_M$. Addestra un modello $y_m(\mathbf{x})$ su ogni $\mathcal{D}_m$.
- **Predizione:** Media (regressione) o votazione a maggioranza (classificazione) degli output dei singoli modelli.

$$y_{bagging}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} y_m(\mathbf{x}) \tag{52}$$

- **Benefici:** Riduce la varianza (particolarmente efficace con modelli ad alta varianza come i Decision Trees).

13.3 Boosting (es. AdaBoost)

Addestra M "weak learners" (modelli semplici, leggermente migliori di una scelta casuale) in sequenza. Ogni learner si concentra sugli errori dei learner precedenti, dando maggior peso agli esempi misclassificati.

- **Training Sequenziale:**
  1. Inizializza pesi $w_n^{(1)} = 1/N$ per tutti gli esempi.
  2. Per $m = 1, \ldots, M$:
     - Addestra un weak learner $y_m(\mathbf{x})$ minimizzando l'errore pesato.
     - Calcola l'errore pesato $\epsilon_m = \frac{\sum w_n^{(m)} \mathbb{I}(y_m(\mathbf{x}_n) \neq t_n)}{\sum w_n^{(m)}}$.
     - Calcola il peso α_m del learner m: $\alpha_m = \ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$.

- Aggiorna i pesi degli esempi: $w_n^{(m+1)} = w_n^{(m)} \exp[\alpha_m \mathbb{I}(y_m(\mathbf{x}_n) \neq t_n)]$.

3. Output il classificatore finale:

$$Y_M(\mathbf{x}) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m y_m(\mathbf{x})\right) \tag{53}$$

- **Benefici:** Riduce il bias (efficace con modelli ad alto bias).

13.4 Bias-Variance Decomposition

L'errore totale di un modello può essere decomposto in Bias, Varianza e Rumore irriducibile:

$$MSE = Bias^2 + Variance + Noise \tag{54}$$

- **Bias:** Errore dovuto a un modello troppo semplice che non può catturare la vera relazione nei dati (underfitting).
- **Varianza:** Sensibilità del modello alle piccole fluttuazioni nel training set (overfitting).
- **Rumore Irriducibile:** Errore intrinseco nei dati, non eliminabile dal modello.

Bagging riduce la varianza, Boosting riduce il bias.

14 Apprendimento Non Supervisionato

Trova strutture nascoste in dati non etichettati.

14.1 K-Means Algorithm

Algoritmo di clustering iterativo per partizionare N osservazioni in K cluster.

- **Input:** Dataset $\mathcal{D} = \{\mathbf{x}_n\}$, numero di cluster K.
- **Output:** I centroidi $\mu_1, \ldots, \mu_K$.
- **Passi Principali:**
  1. **Inizializzazione:** Scegliere K centroidi iniziali (spesso casualmente tra i punti dati).
  2. **Fase di Assegnazione:** Assegnare ogni punto dati al centroide più vicino.
  3. **Fase di Aggiornamento:** Ricalcolare la posizione di ogni centroide come la media di tutti i punti assegnati a quel cluster.
  4. **Terminazione:** Ripetere i passi 2 e 3 fino a quando i centroidi non cambiano più posizione in modo significativo (convergenza).
- **Svantaggi:** Sensibile all'inizializzazione dei centroidi e alla presenza di outlier, tende a formare cluster di forma circolare/sferica.

14.2 Gaussian Mixture Models (GMM)

Assumono che i dati siano generati da una miscela di K distribuzioni Gaussiane.

$$P(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k) \tag{55}$$

I parametri (π_k probabilità a priori del cluster, μ_k media, Σ_k matrice di covarianza) sono stimati con l'algoritmo **Expectation-Maximization (EM)**.

- **E-step (Expectation):** Calcola le "responsabilità" $\gamma(z_k)$, cioè la probabilità che un punto $\mathbf{x}$ appartenga al cluster k.

$$\gamma(z_k) \equiv P(z_k = 1|\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}; \mu_j, \Sigma_j)} \tag{56}$$

- **M-step (Maximization):** Aggiorna i parametri π_k, μ_k, Σ_k usando le responsabilità calcolate.

15 Riduzione della Dimensionalità

Tecniche per ridurre il numero di feature, utili per visualizzazione, riduzione del rumore e miglioramento dell'efficienza computazionale.

15.1 Principal Component Analysis (PCA)

Tecnica lineare che trasforma i dati in un nuovo sistema di coordinate (componenti principali), ordinato per quantità di varianza spiegata.

- **Obiettivo:** Trovare direzioni $\mathbf{u}_i$ (componenti principali) che massimizzano la varianza dei dati proiettati.
- **Soluzione:** Le componenti principali sono gli autovettori della matrice di covarianza $\mathbf{S}$ dei dati, e la varianza corrispondente è data dagli autovalori.
- **Matrice di Covarianza:** $\mathbf{S} = \frac{1}{N}\mathbf{X}^T\mathbf{X}$.
- **High-Dimensional Data ($N < D$):** Si opera sulla matrice $\mathbf{X}\mathbf{X}^T$ (dimensione $N \times N$) invece di $\mathbf{X}^T\mathbf{X}$ (dimensione $D \times D$) per efficienza.

PCA Probabilistica: Una formulazione probabilistica di PCA, che assume un modello generativo lineare Gaussiano con variabili latenti.

15.2 Autoencoders (AE)

Reti neurali autoassociative che apprendono una rappresentazione a bassa dimensione (encoding) di un input, tale da poter ricostruire l'input originale.

- **Architettura:** Composta da un **Encoder** (input $\rightarrow$ rappresentazione latente) e un **Decoder** (rappresentazione latente $\rightarrow$ ricostruzione dell'input). Lo strato nascosto più piccolo è chiamato "bottleneck".
- **Scopo:** Apprendere feature importanti, riduzione non lineare della dimensionalità.
- **Training:** Minimizzare una funzione di costo di ricostruzione (es. MSE) tra input e output.
- **Convolutional Autoencoders:** Per immagini, usano strati convoluzionali e deconvoluzionali (trasposed convolutional layers).
- **Anomaly Detection:** Addestrando l'AE su dati "normali", un'alta errore di ricostruzione per un nuovo input indica un'anomalia.

15.3 Generative Adversarial Networks (GAN)

Un tipo di modello generativo che impara a produrre campioni realistici da una distribuzione di input data, attraverso una competizione tra due reti neurali.

- **Architettura:**

  - **Generatore (G):** Prende un vettore di rumore casuale e produce un campione (es. un'immagine).
  - **Discriminatore (D):** Prende un campione (vero o generato) e cerca di distinguere se è reale o falso.

- **Training (Adversarial Training):** G e D competono: G cerca di ingannare D producendo campioni sempre più realistici, mentre D cerca di migliorare la sua capacità di distinguere i campioni reali da quelli generati.
- **Scopo:** Generazione di dati realistici, aumento di dataset, transfer learning.

16 Markov Decision Processes (MDP) e Reinforcement Learning (RL)

MDP sono framework matematici per modellare il processo decisionale in situazioni dove gli esiti sono parzialmente casuali e sotto il controllo di un decisore. RL si occupa di apprendere policy ottimali in ambienti MDP sconosciuti.

16.1 Proprietà di Markov

La proprietà di Markov afferma che la dinamica futura di un sistema dipende solo dal suo stato corrente, e non dall'intera storia degli stati, azioni e osservazioni precedenti.

- **Processo di Markov:** Un processo che soddisfa la proprietà di Markov.
- **HMM (Hidden Markov Model):** I processi sottostanti sono di Markov, ma lo stato attuale non è direttamente osservabile, solo le osservazioni dipendono dallo stato.
- **MDP (Markov Decision Process):** Lo stato è completamente osservabile e l'agente può prendere decisioni che influenzano la transizione di stato.

MDP vs HMM (Modelli Grafici):

- **HMM:** Stato (X_t) non osservabile, Osservazione (Z_t) dipende da X_t. $X_t \to Z_t$ e $X_{t-1} \to X_t$.
- **MDP:** Stato (X_t) osservabile, Azione (A_t) influenza X_{t+1}. $X_t \to A_t \to X_{t+1}$.
- **POMDP (Partially Observable MDP):** Combinazione, stato non osservabile, azione influenza lo stato, osservazione dipende dallo stato. $X_t \to A_t \to X_{t+1} \to Z_{t+1}$.

16.2 MDP: Modello e Soluzione

Un MDP è definito da $\langle \mathcal{X}, \mathcal{A}, \delta, r \rangle$:

- **Stato ($\mathcal{X}$):** Insieme finito di stati.
- **Azioni ($\mathcal{A}$):** Insieme finito di azioni.
- **Funzione di Transizione (δ):** $P(x'|x,a)$ (probabilistica) o $x' = \delta(x,a)$ (deterministica).
- **Funzione di Ricompensa (r):** $r(x,a,x')$ ricompensa per transizione (x,a,x').

Soluzione di un MDP: Una **policy ottima** $\pi^*(x)$ che massimizza la ricompensa cumulativa scontata.

$$V^\pi(x) \equiv \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots] \tag{57}$$

dove $\gamma \in [0,1]$ è il fattore di sconto.

16.3 Reinforcement Learning (RL)

Apprendere una policy ottima quando l'MDP è sconosciuto (cioè δ e r non sono noti).

- **Funzione di Valore Q ($Q(x, a)$):** Valore atteso di prendere azione a nello stato x e poi seguire una policy ottima.

$$Q(x, a) \equiv \mathbb{E}[r(x, a)] + \gamma \sum_{x'} P(x'|x, a) \max_{a'} Q(x', a') \tag{58}$$

  La policy ottima si deriva da Q: $\pi^*(x) = \arg\max_{a \in \mathcal{A}} Q(x, a)$.

- **Q-Learning:** Algoritmo off-policy per apprendere la funzione Q.

$$\hat{Q}_n(x, a) \leftarrow (1 - \alpha)\hat{Q}_{n-1}(x, a) + \alpha \left[r + \gamma \max_{a'} \hat{Q}_{n-1}(x', a') \right] \tag{59}$$

- **SARSA:** Algoritmo on-policy per apprendere la funzione Q.

$$\hat{Q}_n(x, a) \leftarrow \hat{Q}_{n-1}(x, a) + \alpha \left[r + \gamma \hat{Q}_{n-1}(x', a') - \hat{Q}_{n-1}(x, a) \right] \tag{60}$$

- **Strategie di Esplorazione/Sfruttamento:**
  - **ϵ-greedy:** Con probabilità ϵ esplora (sceglie un'azione casuale), con probabilità $1 - \epsilon$ sfrutta (sceglie l'azione migliore secondo $\hat{Q}$).
  - **Soft-max:** Sceglie azioni con probabilità proporzionale al loro valore Q, dando una chance a tutte le azioni.

- **K-Armed Bandit Problem:** Un caso semplificato di MDP con un solo stato. L'agente deve scegliere tra K azioni con ricompense stocastiche.
- **Policy Gradient Methods:** Invece di apprendere una funzione di valore, apprendono direttamente una policy parametrica $\pi_\theta(x)$ e ottimizzano i parametri θ usando metodi basati sul gradiente.

17 Modelli Lineari per la Regressione

Cercano di approssimare una funzione continua $f : \mathcal{X} \to \mathcal{R}$.

17.1 Modelli Lineari a Funzione Base

$y(\mathbf{x}; \mathbf{w}) = \sum_{j=0}^{M} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$ dove $\phi_j(\mathbf{x})$ sono funzioni base non lineari (es. polinomiali, radiali, sigmoidali) che trasformano lo spazio di input in uno spazio feature. Il modello rimane lineare nei parametri w.

17.2 Maximum Likelihood e Least Squares

Assumendo rumore Gaussiano sull'output $t = y(\mathbf{x}; \mathbf{w}) + \epsilon$, l'ottimizzazione dei parametri w si riduce alla minimizzazione dell'errore quadratico (Least Squares):

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} [t_n - \mathbf{w}^T \phi(\mathbf{x}_n)]^2 \tag{61}$$

La soluzione in forma chiusa è:

$$\mathbf{w}_{ML} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t} \tag{62}$$

dove Φ è la design matrix.

17.3 Regularization (Regolarizzazione)

Tecnica per prevenire l'overfitting, aggiungendo un termine di penalità alla funzione di errore.

$$\min_{\mathbf{w}} E_D(\mathbf{w}) + \lambda E_W(\mathbf{w}) \tag{63}$$

dove $E_W(\mathbf{w})$ penalizza la complessità del modello (es. L_1 o L_2 norm dei pesi).

17.4 Apprendimento Sequenziale (Stochastic Gradient Descent - SGD)

Metodo iterativo per grandi dataset. L'aggiornamento avviene per ogni esempio o per mini-batch: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_S$ dove η è il learning rate. Varianti come SGD con momentum migliorano la convergenza.

18 Strategie per le Domande d'Esame

18.1 Preparazione Generale

- **Formulario a Portata di Mano:** Avere il formulario delle equazioni (come quello che hai generato) ben organizzato e comprensibile è fondamentale.
- **Comprendere i Concetti, Non Solo Memorizzare:** Le domande d'esame non chiedono solo definizioni, ma la comprensione profonda dei "perché" e dei "quando" applicare un certo metodo o perché una certa metrica è più adatta.
- **Esempi Qualitativi e Grafici:** Essere in grado di disegnare un esempio (anche schematico) di dataset, soluzione e problematiche (e.g., overfitting, outlier) è spesso richiesto.
- **Pseudocodice:** Molte domande chiedono di descrivere algoritmi in pseudocodice.

18.2 Approccio alle Domande

1. **Leggi Attentamente la Domanda:** Identifica i verbi chiave (descrivi, formalizza, calcola, discuti, simula).
2. **Identifica il Problema di ML:** È classificazione, regressione, clustering, RL? Quali sono input, output, spazio degli stati/azioni?
3. **Specificare Tutti gli Elementi:** Se richiesto un modello (e.g., MDP, ANN, CNN), elenca e definisci tutti i suoi componenti (stati, azioni, reward, pesi, bias, funzioni di attivazione, kernel, strati, etc.) e le loro dimensioni.
4. **Formalizzazione Matematica:** Scrivi le formule rilevanti, spiegando il significato di ogni termine.
5. **Spiegazione Algoritmica:** Descrivi i passi dell'algoritmo (anche in pseudocodice), specificando input, output e condizioni di terminazione.
6. **Discussione Critica:** Se richiesto, analizza i pro e i contro di una scelta, discuti le assunzioni (e.g., Naive Bayes), le limitazioni (e.g., K-Means con outlier), o le implicazioni (e.g., overfitting).
7. **Esempi Numerici/Grafici:** Se una domanda chiede un esempio numerico, cerca di usarne uno semplice che evidenzi il concetto richiesto. Se chiede un grafico, assicurati che sia chiaro e etichettato correttamente.

18.3 Temi Ricorrenti dagli Esami Forniti

- **Fondamentali:** Definizione di ML, tipi di apprendimento, bias-variance.
- **Valutazione:** Matrice di confusione, metriche (accuracy, precision, recall, F1, TPR, TNR, FPR, FNR), K-Fold Cross Validation, overfitting e mitigazione.
- **Decision Trees:** ID3, Information Gain, Entropy, pruning, gestione attributi continui/molti valori.
- **Bayesiano:** MAP, ML, Bayes Optimal Classifier (BOC), Naive Bayes (assunzioni, stima probabilità, classificazione testi).
- **Lineari Classificazione/Regressione:** Modelli lineari, Perceptron (modello, training), SVM (max margin, soft margin, support vectors, why preferred), Fisher's Linear Discriminant.
- **Kernel Methods:** Kernel trick, Gram matrix, kernel function (tipologie), kernelized linear models/SVM.
- **ANN/CNN:** Architettura (strati, dimensioni, parametri), funzioni di attivazione (ReLU, Sigmoid, Tanh), backpropagation, SGD (momentum), loss functions (cross-entropy, MSE), tecniche di regolarizzazione (dropout, early stopping, data augmentation, parameter norm penalties).
- **Unsupervised:** K-Means (algoritmo, pro/contro, inizializzazione), GMM (EM algorithm).
- **Dimensionality Reduction:** PCA (variance maximization, error minimization, high-dim data), Autoencoders (architettura, scopo, anomaly detection), GANs.
- **Reinforcement Learning:** MDP (modello, stati, azioni, transizioni, reward), Q-Learning/SARSA (algoritmi, esplorazione/sfruttamento), k-armed bandit.